Tecnológico Nacional de México Instituto Tecnológico de Tijuana

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

Carrera

Ingeniería en Sistemas Computacionales

Materia

Patrones de diseño

Título Actividad

Actividad 2

Unidad

Unidad 2

Nombre y No. Control del Alumno

Brandon Orozco Hernandez 21212577

Nombre del Maestro

MARIBEL GUERRERO LUIS

Instrucción

El **Singleton** es un patrón de diseño cuyo objetivo es garantizar que una clase solo pueda crear **una única instancia durante toda la ejecución del programa**. Este patrón se utiliza cuando se necesita trabajar con un recurso compartido, como por ejemplo una central de emergencias, una conexión a una base de datos o alguna configuración global que debe ser única.

Atributo estático de la instancia

private static Central_911 _instance;

¿Qué significa cada palabra?

- **private** → Indica que esta variable no puede ser accedida directamente desde otras clases; solo la propia clase puede manipularla.

- **static** → Significa que la variable pertenece a la clase en sí y no a un objeto específico. Existe incluso si todavía no se ha creado ningún objeto.

- **Central_911** → Es el tipo de dato de la variable, es decir, almacenará una referencia a un objeto de tipo Central_911.

- **_instance** → Es la variable que guarda la única instancia que se creará de la central.

Constructor

```
private Central_911()
{
   Central = "Central 911";
}
```

¿Qué significa cada parte?

- **private** → Impide que otras clases puedan crear un objeto usando new Central_911(). Si alguien intenta hacerlo desde fuera, el programa marcará un error.

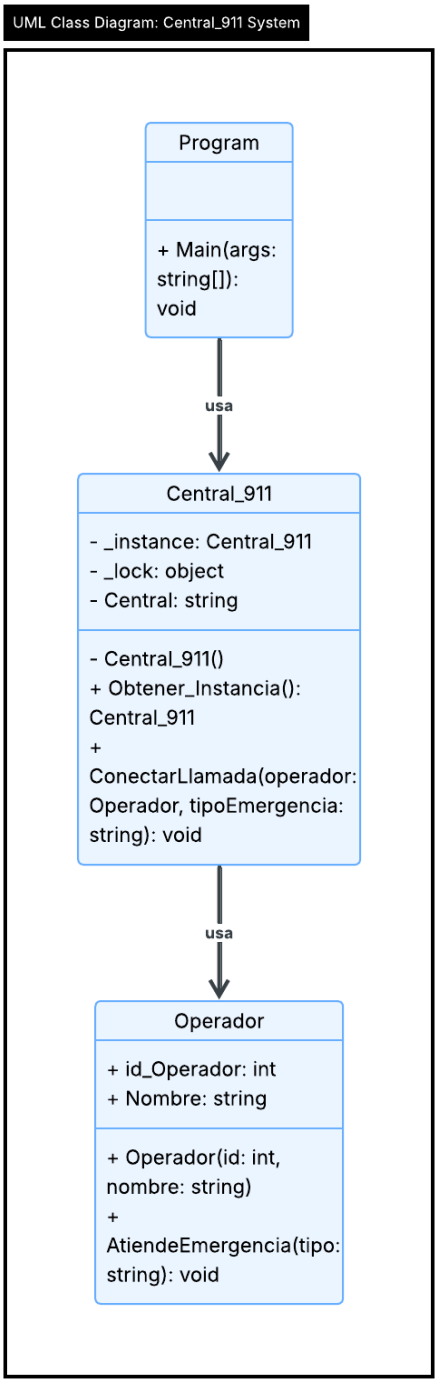
- **Central_911()** → Es el **constructor de la clase**, es decir, el método que se ejecuta automáticamente cuando se crea el objeto.

- **Central = "Central 911"** → Aquí se asigna el nombre a la central en el momento en que se crea la instancia.

¿Por qué el constructor es privado?

Esto es fundamental para que funcione el patrón **Singleton**. Al hacer el constructor privado, se evita que alguien cree múltiples objetos por accidente.
De esta manera, la **única forma de obtener la instancia** es mediante el método Obtener_Instancia(), el cual se encarga de verificar si ya existe una instancia o si debe crearla.

Diagrama UML



¿Por qué no se pueden crear múltiples objetos?

La razón por la que no se pueden crear múltiples objetos es porque el patrón Singleton está diseñado para permitir únicamente una sola instancia de la clase durante toda la ejecución del programa. Esto se logra principalmente haciendo que el constructor sea privado, lo que impide que otras clases puedan crear objetos usando la palabra new.